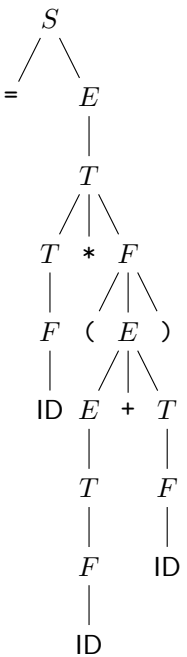
コンパイラ (7)

2026/10/23

抽象構文

ID=ID∗(ID+ID)

$$
\begin{aligned}
E &\rightarrow E + T \\
E &\rightarrow T \\
T &\rightarrow T * F \\
T &\rightarrow F \\
F &\rightarrow \mathrm{ID} \\
F &\rightarrow (E) \\
S &\rightarrow \mathrm{ID} = E
\end{aligned}
$$



$$
\begin{aligned}
E &\rightarrow E + E \\
E &\rightarrow E * E \\
E &\rightarrow \mathrm{ID} \\
S &\rightarrow \mathrm{ID} = E
\end{aligned}
$$

具体構文および構文木　　抽象構文

抽象構文：具体構文から意味に関わる部分を抽出

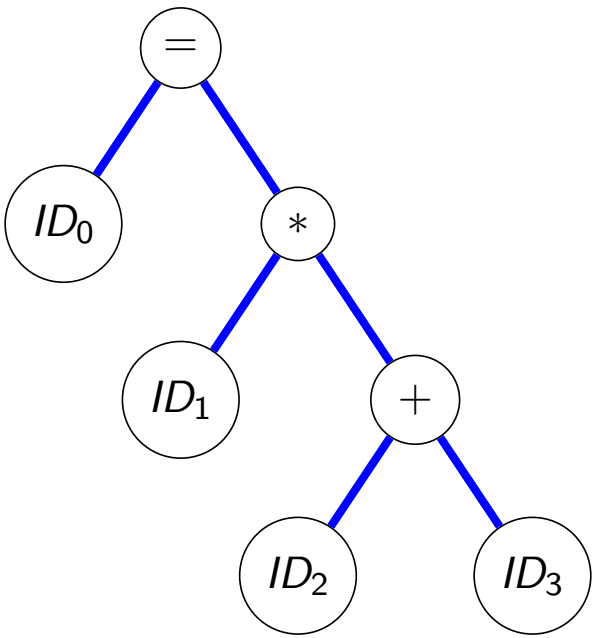
抽象構文木

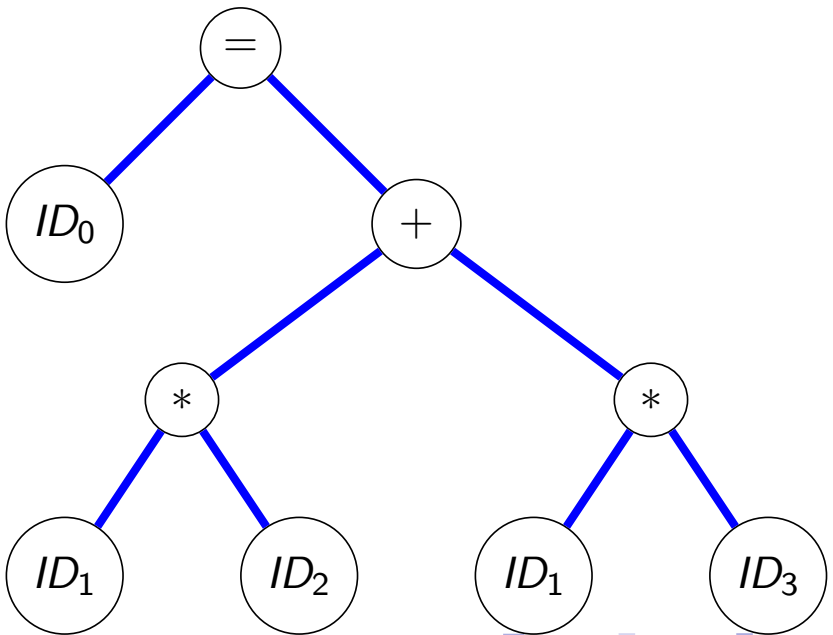
解析木からコード生成に必要な部分を残して変形したもの

内部節点=演算子、葉=オペランド

構文主導型変換によって中間コードを生成する際に便利

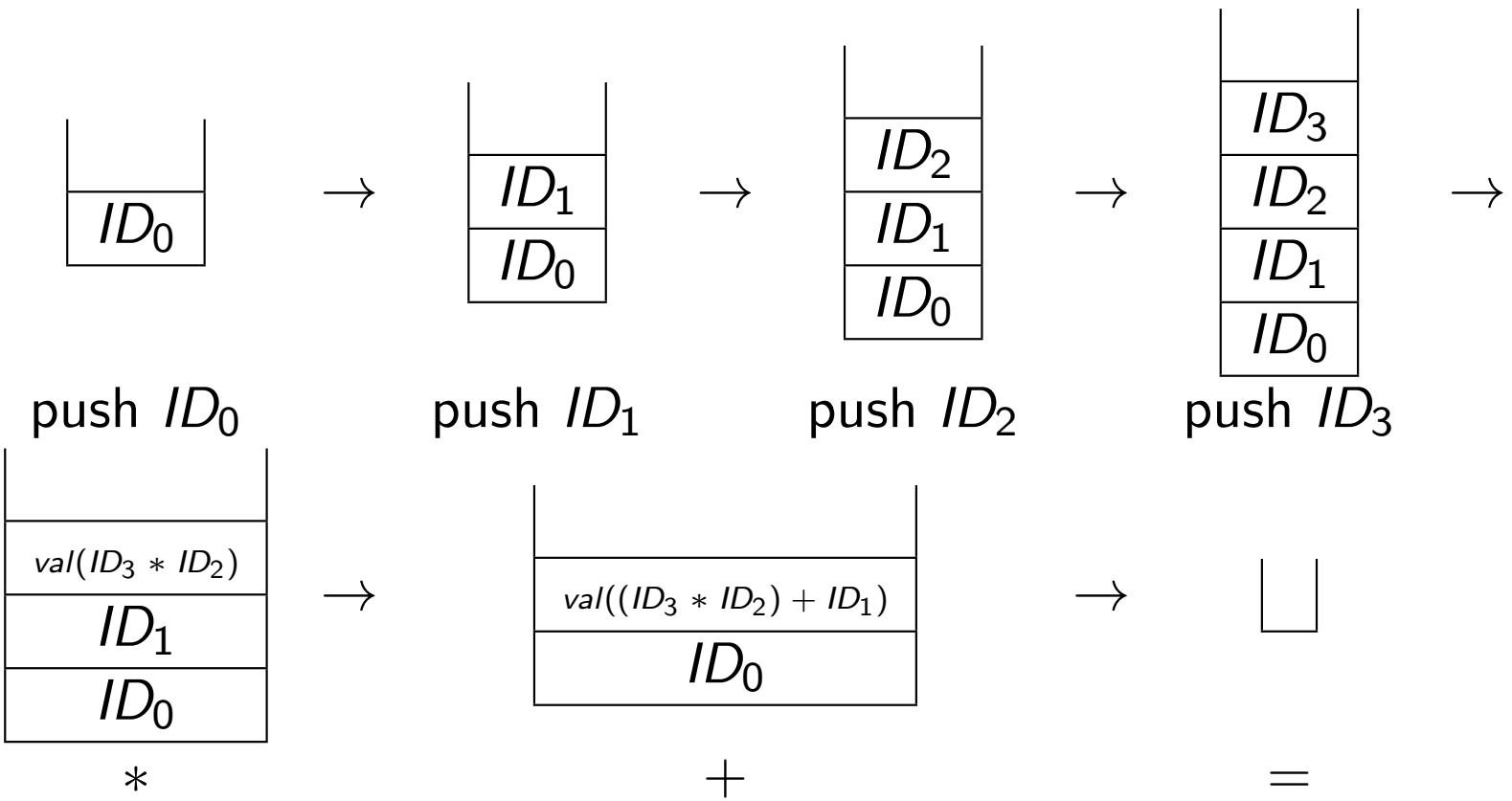
等価な意味を持つ抽象構文木=最適化に利用





逆ポーランド記法

抽象構文木の後置記法: スタックを用いた実行に対応



3番地コード

$C = A\theta B$ $+,-,*,/,$and,or,$>,>=,=,!=,<=,<$

$B = \theta A$ $-,$not

$B = A$

Jump on condition

`if` A `goto` L

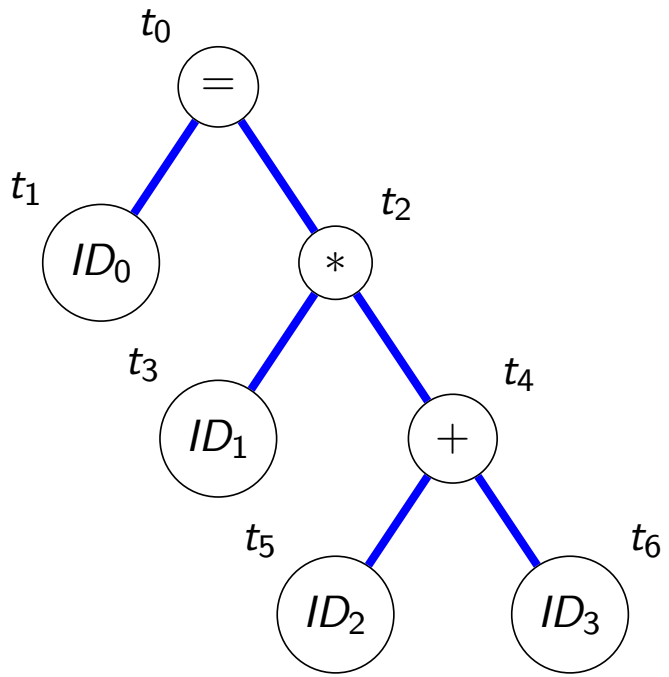
`if` $notA$ `goto` L

Unconditional Jump

`goto` L

抽象構文木からの３番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$



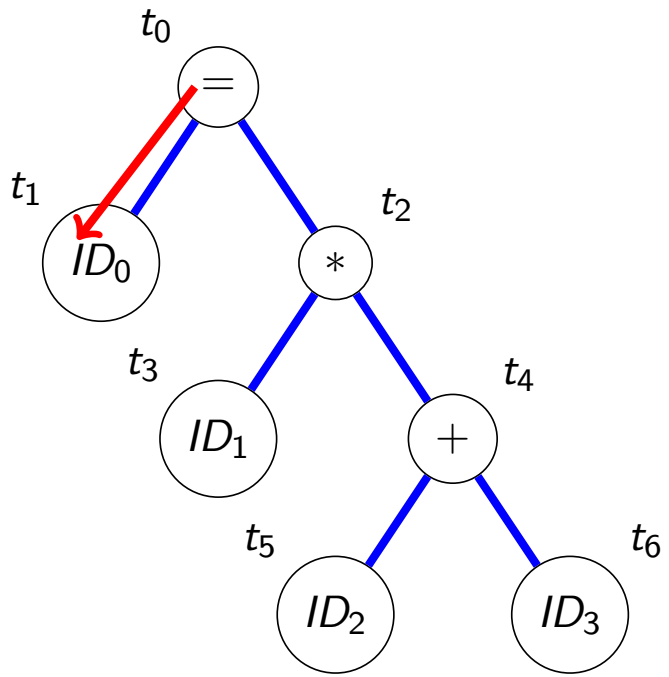
$$t_1 = ID_0$$
$$t_3 = ID_1$$
$$t_5 = ID_2$$
$$t_6 = ID_3$$
$$t_4 = t_5 + t_6$$
$$t_2 = t_3 * t_4$$
$$t_0 = (t_1 = t_2)$$

抽象構文木に一時変数 t_i を割り当てる

抽象構文木からの３番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$t_1 = ID_0$
$t_3 = ID_1$
$t_5 = ID_2$
$t_6 = ID_3$
$t_4 = t_5 + t_6$
$t_2 = t_3 * t_4$
$t_0 = (t_1 = t_2)$

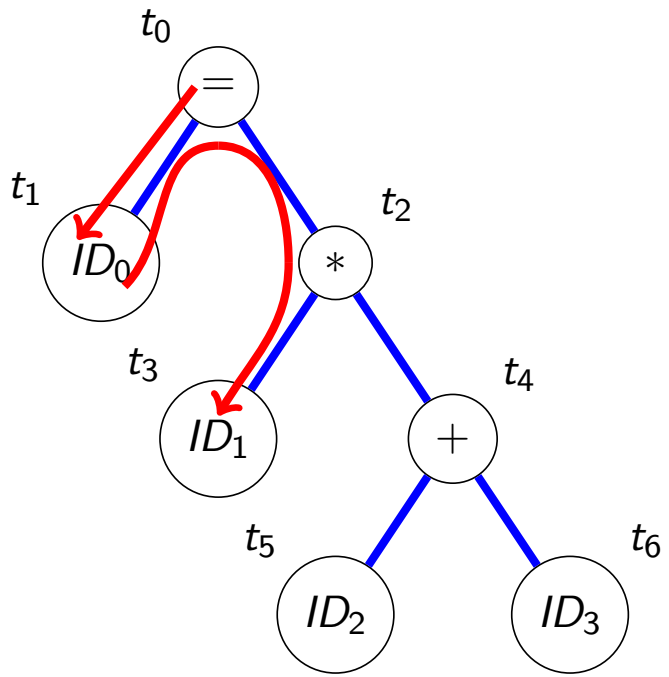


ポストオーダーでトレースし，$t_1 = ID_0$ を生成

抽象構文木からの３番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$t_1 = ID_0$
$t_3 = ID_1$
$t_5 = ID_2$
$t_6 = ID_3$
$t_4 = t_5 + t_6$
$t_2 = t_3 * t_4$
$t_0 = (t_1 = t_2)$

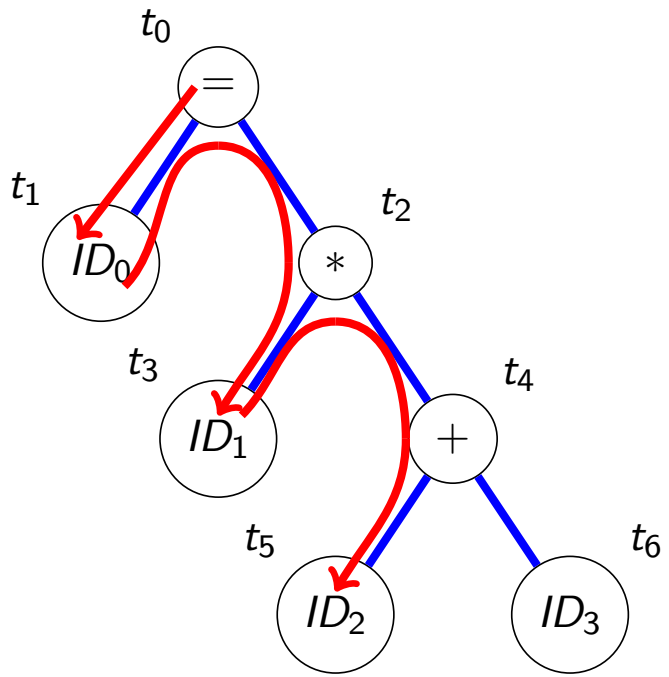


ポストオーダーでトレースし，$t_3 = ID_1$ を生成

抽象構文木からの３番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$t_1 = ID_0$
$t_3 = ID_1$
$t_5 = ID_2$
$t_6 = ID_3$
$t_4 = t_5 + t_6$
$t_2 = t_3 * t_4$
$t_0 = (t_1 = t_2)$

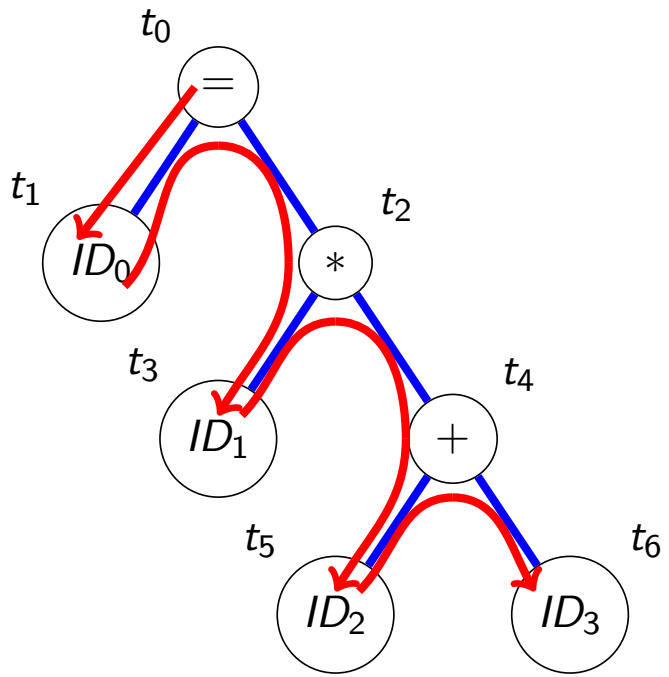


ポストオーダーでトレースし，$t_5 = ID_2$ を生成

抽象構文木からの３番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$t_1 = ID_0$
$t_3 = ID_1$
$t_5 = ID_2$
$t_6 = ID_3$
$t_4 = t_5 + t_6$
$t_2 = t_3 * t_4$
$t_0 = (t_1 = t_2)$

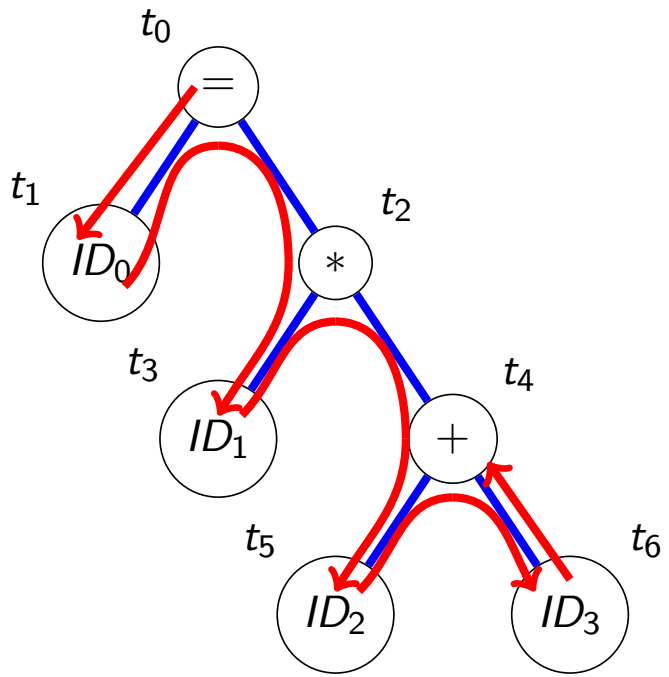


ポストオーダーでトレースし，$t_6 = ID_3$ を生成

抽象構文木からの３番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$t_1 = ID_0$
$t_3 = ID_1$
$t_5 = ID_2$
$t_6 = ID_3$
$t_4 = t_5 + t_6$
$t_2 = t_3 * t_4$
$t_0 = (t_1 = t_2)$

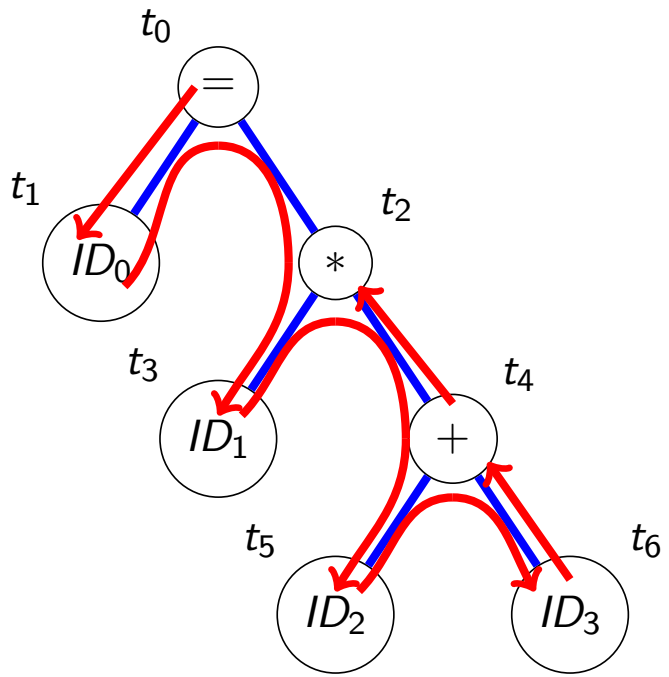


ポストオーダーでトレースし，$t_4 = t_5 + t_6$ を生成

抽象構文木からの３番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$
$$t_3 = ID_1$$
$$t_5 = ID_2$$
$$t_6 = ID_3$$
$$t_4 = t_5 + t_6$$
$$t_2 = t_3 * t_4$$
$$t_0 = (t_1 = t_2)$$

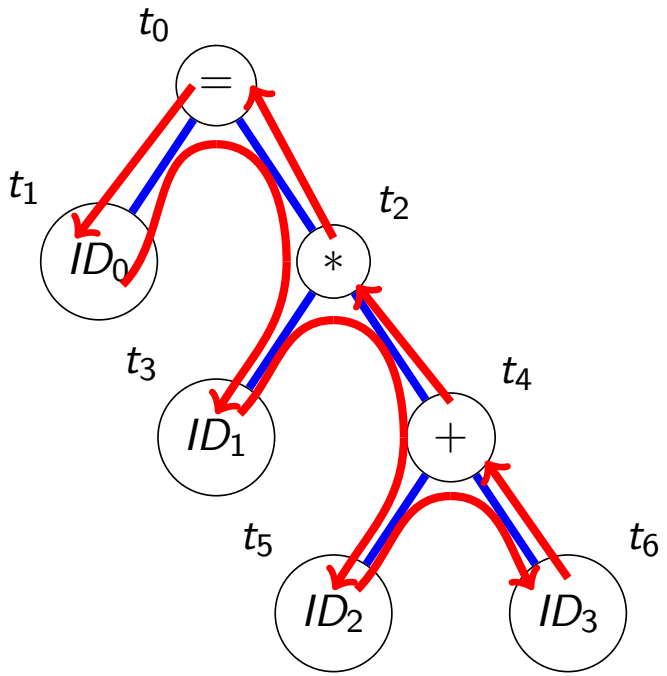


ポストオーダーでトレースし，$t_2 = t_3 * t_4$ を生成

抽象構文木からの３番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$\begin{aligned}
t_1 &= ID_0 \\
t_3 &= ID_1 \\
t_5 &= ID_2 \\
t_6 &= ID_3 \\
t_4 &= t_5 + t_6 \\
t_2 &= t_3 * t_4 \\
t_0 &= (t_1 = t_2)
\end{aligned}$$



ポストオーダーでトレースし，$t_0 = (t_1 = t_2)$ を生成

手続き呼び出しの３番地コード

```
int fib(int x) {
  if (x=0) {
    return 0;
  } else if (x=1) {
    return 1;
  } else {
    return fib(x-2)+fib(x-1);
  }
}

print fib(4);
```

```
  define @fib(x) {
    t1 := x
    t2 := (x=0)
    if not t1 goto l1
    return 0
l1: t3 := (x=1)
    if not t2 goto l2
    return 1
l2: t4 := t1 - 2
    t5 := call fib(t4)
    t6 := t1 - 1
    t7 := call fib(t6)
    t8 := t5 + t7
    return t8
}

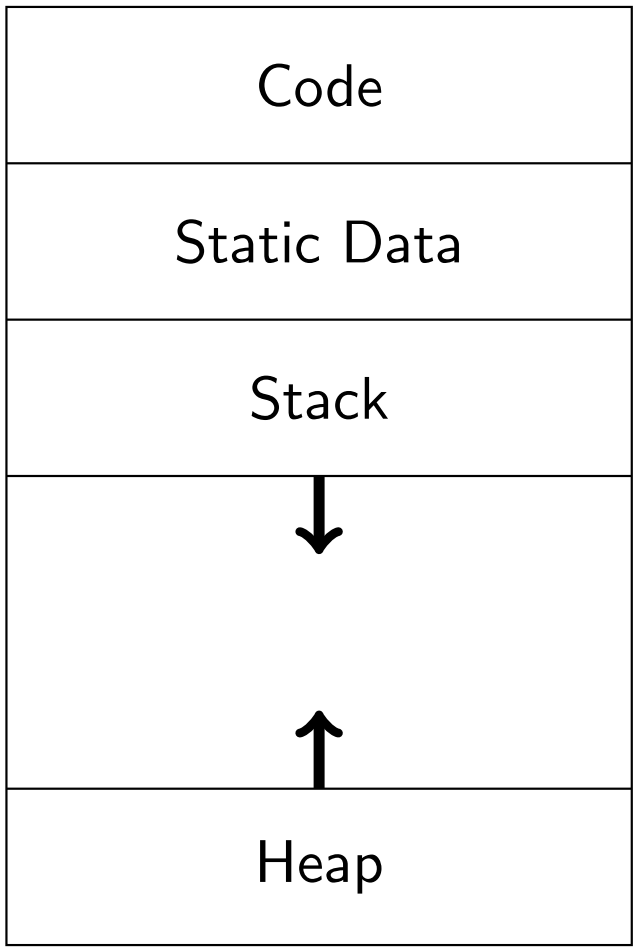
define @main() {
    t1 := 4
    t2 := call fib(t1)
    print t2
}
```

定義：`define @関数名 (仮引数リスト)`
呼出：`call @関数名 (実引数リスト)`

実行時環境

メモリ領域の割り当て方

- 大域変数=ヒープ領域 (プロセス内のメモリ領域) にわりあてる。
  `malloc,free`
  演習のコンパイラでは、スタック領域の底にわりあてる (pop しない)。
- 局所変数=スタックにわりあてる
- 関数・手続の引数のわりあて (特殊な局所変数)



静的割当てと動的割当て

静的割当て: 実行の最初から最後までメモリ領域が変化しない

動的割り当て：メモリ領域が実行時に決定
実行終了時に領域が不要になる

```
int a;

int func(int x) {
    int y;
    y = x * x;
    return (y+y);
}

main() {
    int b=1;
    a=2;
    printf("%d\n",func(a)+b);
}
```

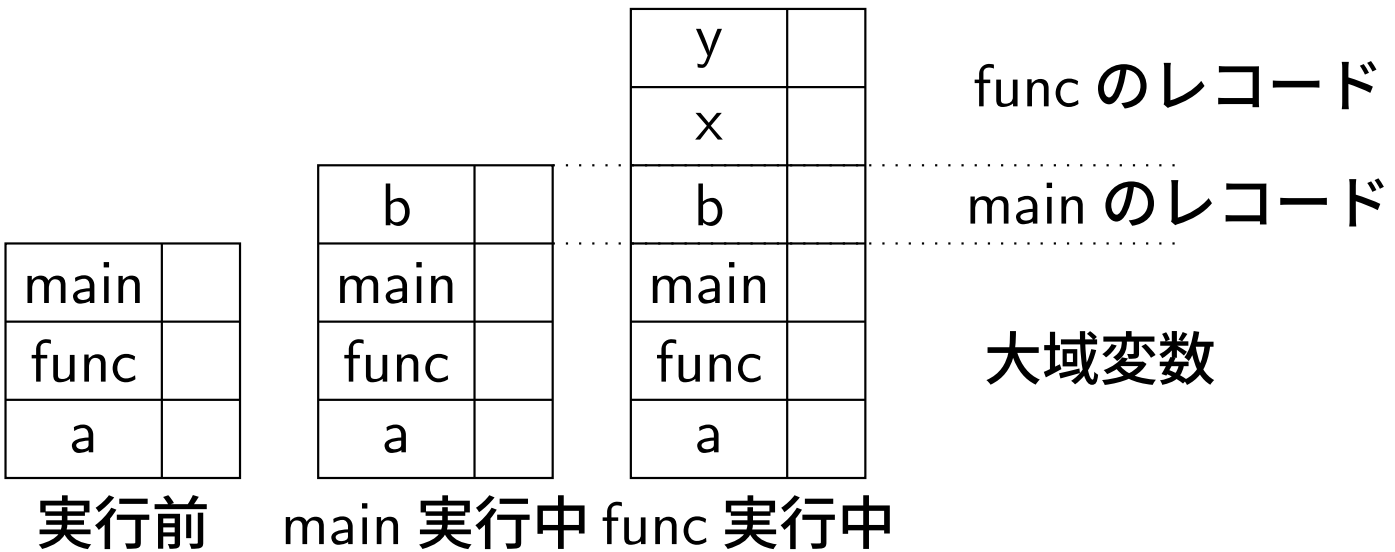
静的割当て: a,func,main

動的割当て: x,y,b

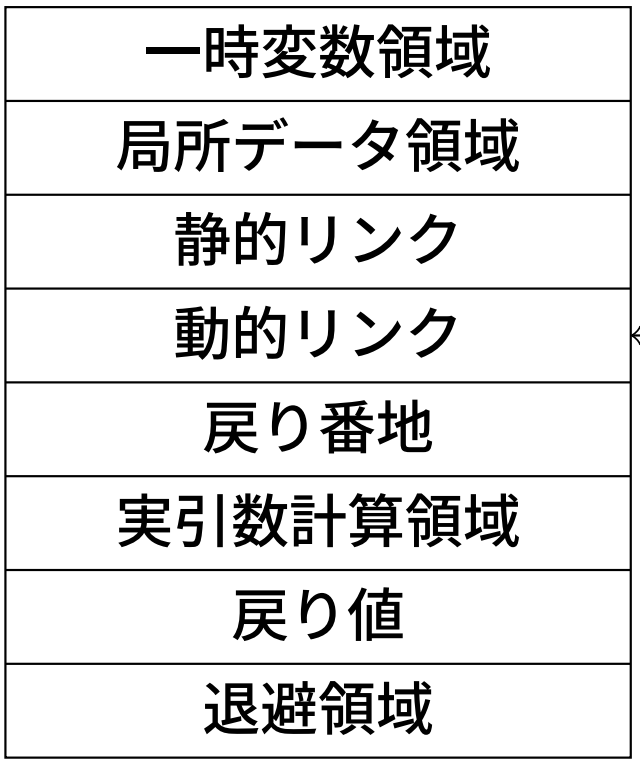
駆動レコード

手続き/関数の一回の実行に必要な局所情報

PC(Program Counter)、返り値、参照可能なシンボルの集合



駆動レコードの構成



関数内での計算の一時領域

局所宣言された変数の領域

次に外へ参照すべきレコード

Return 後に復元すべきレコード

Return 後に再開する PC

呼出の際に引数受け渡しに使う

関数の計算結果を格納

呼出の際の一時領域

変数の参照

```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
    y=func(x-1)*2;
    return(a+y);
  }
}
void main() {
  a=2;
  printf("func(3)=%d",
    func(3));
}
```

?

a=?

変数の参照

```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
    y=func(x-1)*2;
    return(a+y);
  }
}
void main() {
→   a=2;
  printf("func(3)=%d",
    func(3));
}
```

?

a=2

変数の参照

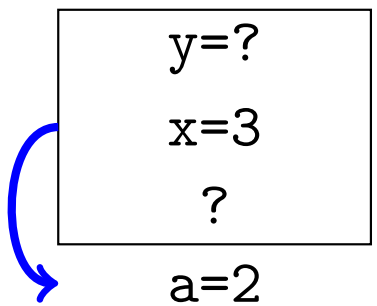
```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
    y=func(x-1)*2;
    return(a+y);
  }
}
void main() {
  a=2;
→ printf("func(3)=%d",
    func(3));
}
```

?

a=2

変数の参照

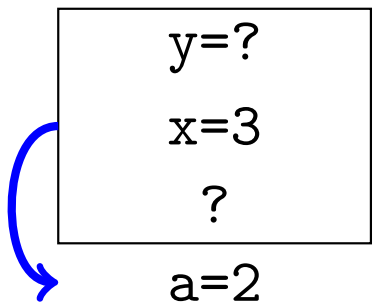
```
  int a;
  int func(int x) {
    int y;
→   if(x==1) {return(a);}
    else {
      y=func(x-1)*2;
      return(a+y);
    }
  }
  void main() {
    a=2;
    printf("func(3)=%d",
      func(3));
  }
```



静的リンク

変数の参照

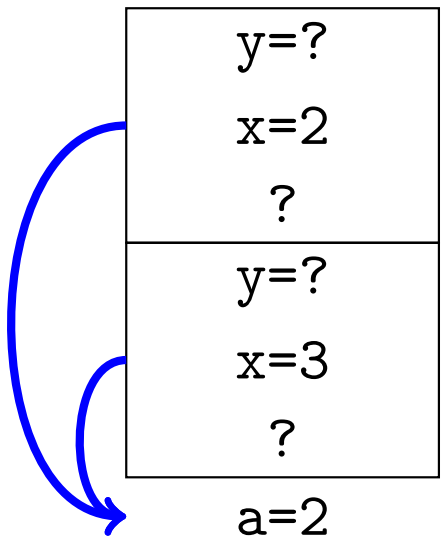
```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
→   y=func(x-1)*2;
    return(a+y);
  }
}
void main() {
  a=2;
  printf("func(3)=%d",
    func(3));
}
```



静的リンク

変数の参照

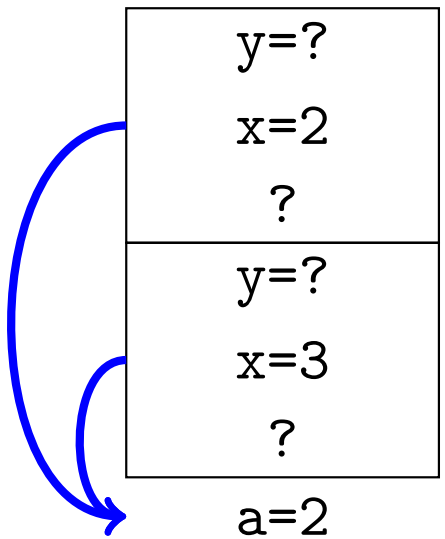
```
  int a;
  int func(int x) {
    int y;
→   if(x==1) {return(a);}
    else {
      y=func(x-1)*2;
      return(a+y);
    }
  }
  void main() {
    a=2;
    printf("func(3)=%d",
      func(3));
  }
```



静的リンク

変数の参照

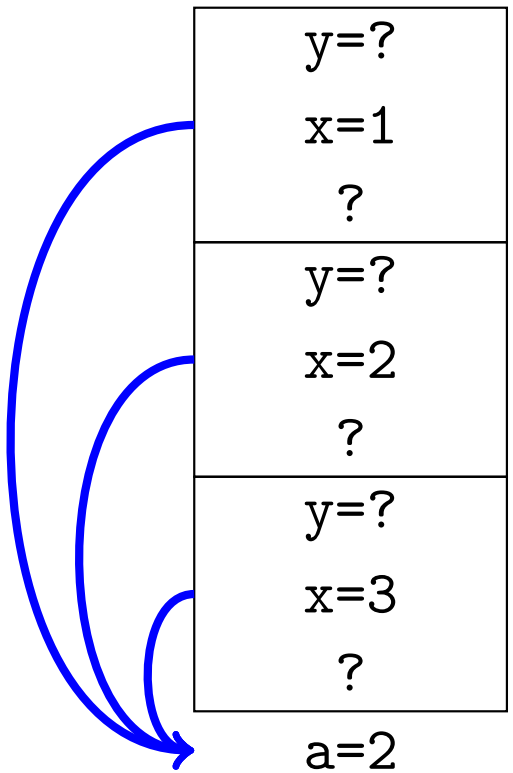
```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
→   y=func(x-1)*2;
    return(a+y);
  }
}
void main() {
  a=2;
  printf("func(3)=%d",
    func(3));
}
```



静的リンク

変数の参照

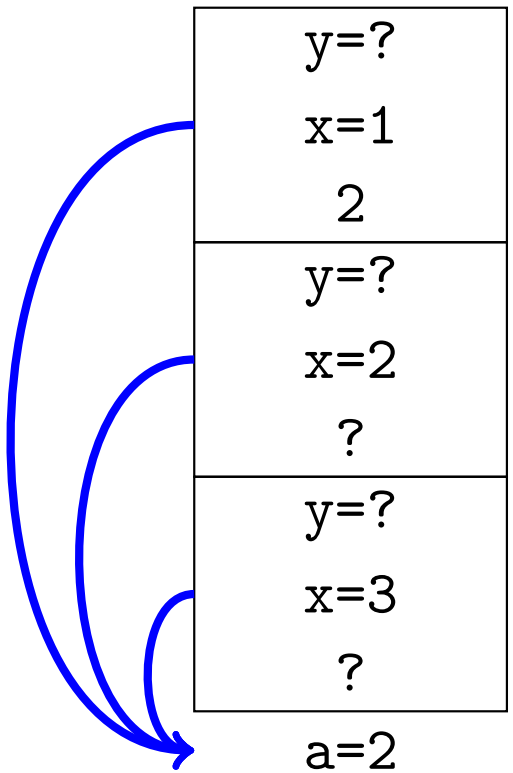
```
  int a;
  int func(int x) {
    int y;
→   if(x==1) {return(a);}
    else {
      y=func(x-1)*2;
      return(a+y);
    }
  }
  void main() {
    a=2;
    printf("func(3)=%d",
      func(3));
  }
```



静的リンク

変数の参照

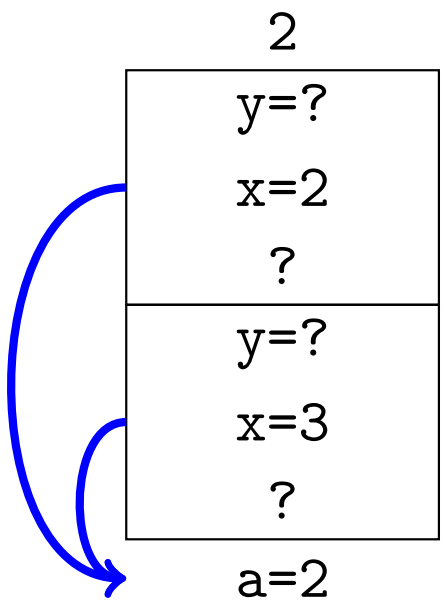
```
  int a;
  int func(int x) {
    int y;
→   if(x==1) {return(a);}
    else {
      y=func(x-1)*2;
      return(a+y);
    }
  }
  void main() {
    a=2;
    printf("func(3)=%d",
      func(3));
  }
```



静的リンク

変数の参照

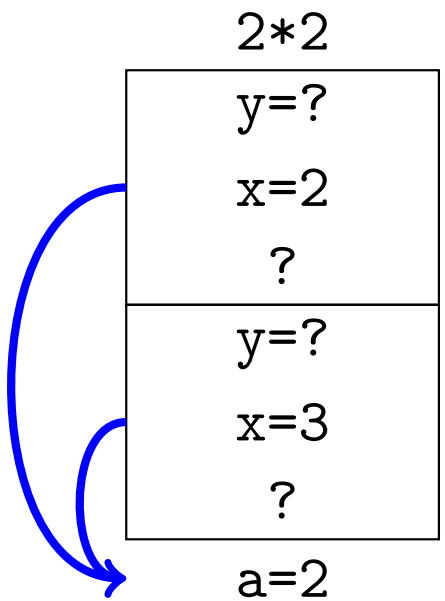
```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
→   y=func(x-1)*2;
    return(a+y);
  }
}
void main() {
  a=2;
  printf("func(3)=%d",
    func(3));
}
```



静的リンク

変数の参照

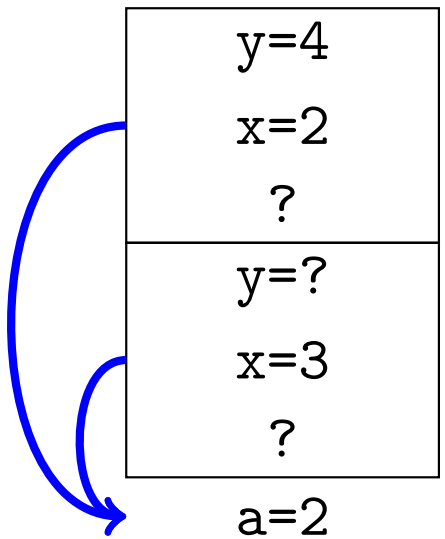
```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
→   y=func(x-1)*2;
    return(a+y);
  }
}
void main() {
  a=2;
  printf("func(3)=%d",
    func(3));
}
```



静的リンク

変数の参照

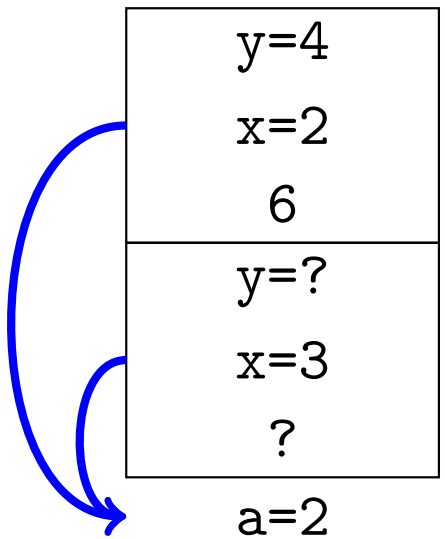
```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
→   y=func(x-1)*2;
    return(a+y);
  }
}
void main() {
  a=2;
  printf("func(3)=%d",
    func(3));
}
```



静的リンク

変数の参照

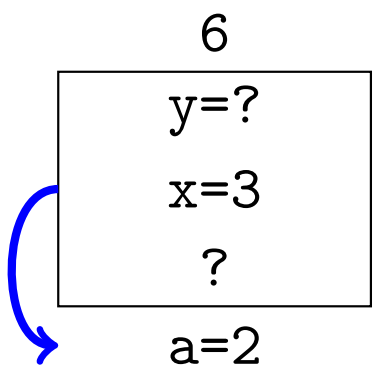
```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
    y=func(x-1)*2;
→   return(a+y);
  }
}
void main() {
  a=2;
  printf("func(3)=%d",
    func(3));
}
```



静的リンク

変数の参照

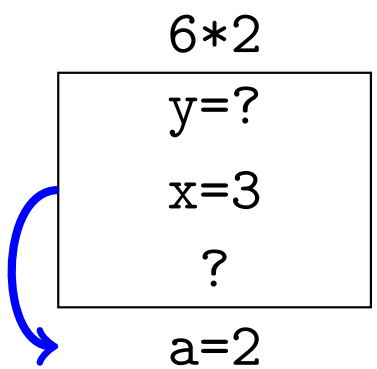
```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
→   y=func(x-1)*2;
    return(a+y);
  }
}
void main() {
  a=2;
  printf("func(3)=%d",
    func(3));
}
```



静的リンク

変数の参照

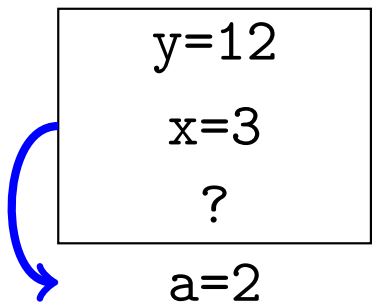
```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
→   y=func(x-1)*2;
    return(a+y);
  }
}
void main() {
  a=2;
  printf("func(3)=%d",
    func(3));
}
```



静的リンク

変数の参照

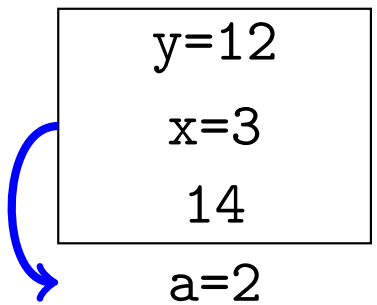
```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
→   y=func(x-1)*2;
    return(a+y);
  }
}
void main() {
  a=2;
  printf("func(3)=%d",
    func(3));
}
```



静的リンク

変数の参照

```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
    y=func(x-1)*2;
→   return(a+y);
  }
}
void main() {
  a=2;
  printf("func(3)=%d",
    func(3));
}
```



静的リンク

変数の参照

```
int a;
int func(int x) {
  int y;
  if(x==1) {return(a);}
  else {
    y=func(x-1)*2;
    return(a+y);
  }
}
void main() {
→   a=2;
  printf("func(3)=%d",
    func(3));
}
```

```
14
a=2
```

変数の参照/記憶位置の決定

局所変数: フレームポインタからの相対位置

大域変数: ヒープ内のアドレス

記号表にはスタックフレームレジスタからの相対番地=何番目に宣言された変数でどういった型をもつか。
型情報=確保する領域量 (実習の場合は整数のみ)
関数の引数うけわたしは局所変数の宣言+代入

動作意味

$$\langle \{S, E, T, F\}, \{+, *, (,), n\}, P, S \rangle$$

$$\left\{ \begin{array}{ll} S \to E\#, & T \to F, \\ E \to E + T, & F \to (\ E\ ), \\ E \to T, & F \to n, \\ T \to T * F & \end{array} \right\}$$

各生成規則は値を計算する意味を持つ.

$E \to E + T$: 右辺の E の値と T の値を加算した値を左辺の E の値とする.

$\Rightarrow$ '+' に加算の意味を与える.

構文に従った動作意味

$S \to E\#$	$var(S) = var(E)$
$E \to E_1 + T$	$var(E) = var(E_1) + var(T)$
$E \to T$	$var(E) = var(T)$
$T \to T_1 * F$	$var(T) = var(T_1) \times var(F)$
$T \to F$	$var(T) = var(F)$
$F \to (E)$	$var(F) = var(E)$
$F \to i$	$temp = val(i); var(F) = temp$

構文主導型変換

インタープリタ： 生成規則に対応する動作をする．

コンパイラ： 生成規則に対応した目的コード（中間コード）を生成する．

構文主導型変換

CF 文法の各生成規則に対してコード生成ルーチンをわりあてる。

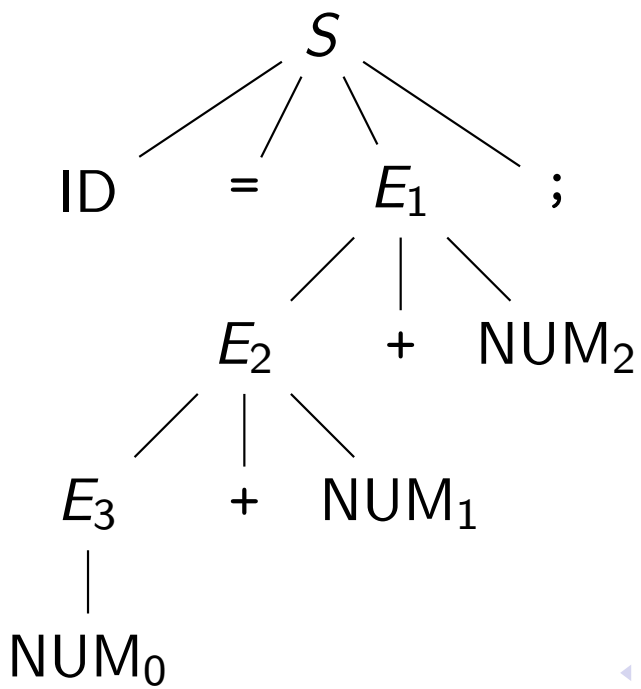
- newTemp()
  変数 T の領域を確保して、記号番地または一時変数へのポインタを返す。
- newLabel()
  新たなラベル名を返す。
- generate(中間コード)
  中間コードを出力ファイルに出力する。

n 回目に newTemp() を呼びだしたとき、生成される変数 tn
m 回目に newLabel() を呼びだしたとき、生成されるラベル Lm

構文主導型変換の基礎

$$
\begin{aligned}
S &\rightarrow \mathrm{ID}= E\ ; \\
E &\rightarrow E^{(1)} + \mathtt{NUM} \\
E &\rightarrow \mathtt{NUM}
\end{aligned}
$$

$$\mathrm{ID} = \mathrm{NUM}_0 + \mathrm{NUM}_1 + \mathrm{NUM}_2\ ;$$



中間コード (3 番地コード) 生成

生成規則	出力される 3 番地コード
$E_3 \to \texttt{NUM}_0$	$\texttt{t3} = \texttt{NUM}_0.val$
$E_2 \to E_3 \texttt{+} \texttt{NUM}_1$	$\texttt{t2} = \texttt{t3} + \texttt{NUM}_1.val$
$E_1 \to E_2 \texttt{+} \texttt{NUM}_2$	$\texttt{t1} = \texttt{t2} + \texttt{NUM}_2.val$
$S \to \texttt{ID} \texttt{=} E_1 \texttt{;}$	$\texttt{ID}.name = \texttt{t1}$

$E \to \texttt{NUM}$ $\{E.var\texttt{:=newTemp();}$
$\texttt{generate(‘}E.var\texttt{=NUM}.val\texttt{’)}$

$E \to E^{(1)} \texttt{+} \texttt{NUM}$ $\{E.var\texttt{:=newTemp();}$
$\texttt{generate(‘}E.var\texttt{=}E^{(1)}.var + \texttt{NUM}.val\texttt{’)}\}$

$S \to \texttt{ID} \texttt{=} \texttt{E} \texttt{;}$ $\{\texttt{generate(‘ID}.name\texttt{=}E.var\texttt{’)}\}$

IF 文の中間コード生成

$$\text{文} \rightarrow \text{if}\ '('\ \text{式}\ ')'\ \text{文}^{(1)}$$

1. '式' のコードを生成 t1 に結果収納
2. if not t1 goto l1
3. '文'$^{(1)}$ のコードを生成
4. l1:

IF 文の中間コード生成

$$\text{文} \rightarrow \text{if}\ '('\ \text{式}\ ')'\ \text{文}^{(1)}\ \text{else}\ \text{文}^{(2)}$$

1. '式' のコードを生成 t1 に結果収納
2. if not t1 goto l1
3. '文'$^{(1)}$ のコードを生成
4. goto l2
5. l1:
6. '文'$^{(2)}$ のコードを生成
7. l2:

while 文の中間コード生成

$$\text{文} \rightarrow \text{while}\ '('\ \text{式}\ ')'\ \text{文}^{(1)}$$

1. l1:
2. ' 式' のコードを生成 t1 に結果収納
3. if not t1 goto l2
4. ' 文'$^{(1)}$ のコードを生成
5. goto l1
6. l2:

構文主導型変換によるコード生成

文 →	if (	{ 文.label:=newLabel();}
	式	(式のコード)
	)	{generate('if_not 式.var goto 文.label')}
	文 (1)	(文 (1) のコード){ 文 (1).label:=newLabel();
		generate('goto 文 (1).label')}
	else	{generate('label 文.label')}
	文 (2)	{generate('label 文 (1).label')}

文 →	while (	{ 文.label1:=newLabel();
		generate('label 文.label1')}
	式	(式のコード)
	)	{ 文.label2:=newLabel();
		generate('if_not 式.var goto 文.label2')}
	文 (1)	(文のコード)
		{generate('goto 文.label1');
		generate('label 文.label2')}

PDT を用いた構文主導型変換

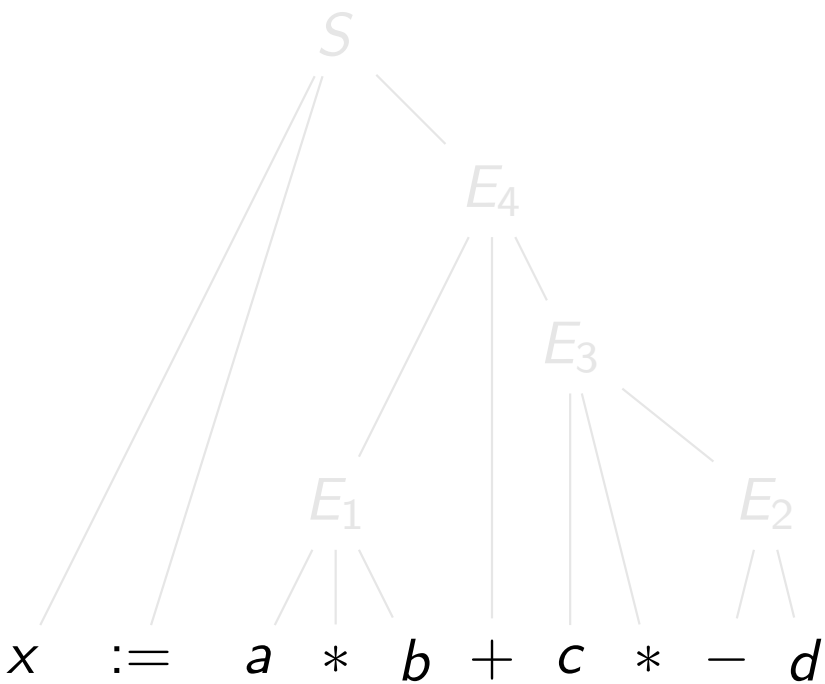
コード生成を行うタイミング=還元操作

$$
\begin{array}{lll}
1: & E \to E^{(1)} \texttt{ + } A & \{E.var{:=}\texttt{newLabel}();\\
 & & \texttt{generate}(\text{`}E.var{:=}E^{(1)}.var + A.var')\}\\
2: & E \to A & \{E.var{:=}\texttt{newTemp}();\\
 & & \texttt{generate}(\text{`}E.var{:=}A.var')\}\\
3: & A \to A^{(1)} \texttt{ * NUM} & \{A.var{:=}\texttt{newTemp}();\\
 & & \texttt{generate}(\text{`}A.var{:=}A^{(1)}.var \times \texttt{NUM}.val')\}\\
4: & A \to \texttt{NUM} & \{A.var := \texttt{newTemp}();\\
 & & \texttt{generate}(\text{`}A.var{:=}\texttt{NUM}.val')\}
\end{array}
$$

$\vdash$	S_0	3+4*5#		
$\vdash$	$S_0 n S_6$	+4*5#		$s6$
$\vdash$	$S_0 T S_7$	+4*5#	t1=3	$r4$
$\vdash$	$S_0 E S_1$	+4*5#	t2=3	$r2$
$\vdash$	$S_0 E S_1$+S_2	4*5#		$s2$
$\vdash$	$S_0 E S_1$+$S_2 n S_6$	*5#		$s6$
$\vdash$	$S_0 E S_1$+$S_2 T S_3$	*5#	t3=5	$r4$
$\vdash$	$S_0 E S_1$+$S_2 T S_3$*S_4	5#		$s4$
$\vdash$	$S_0 E S_1$+$S_2 T S_3$*$S_4 n S_5$	#		$s5$
$\vdash$	$S_0 E S_1$+$S_2 T S_3$	#	t4=t3×5	$r3$
$\vdash$	$S_0 E S_1$	#	t5=t2+t4	$r1$
$\vdash$	acc			

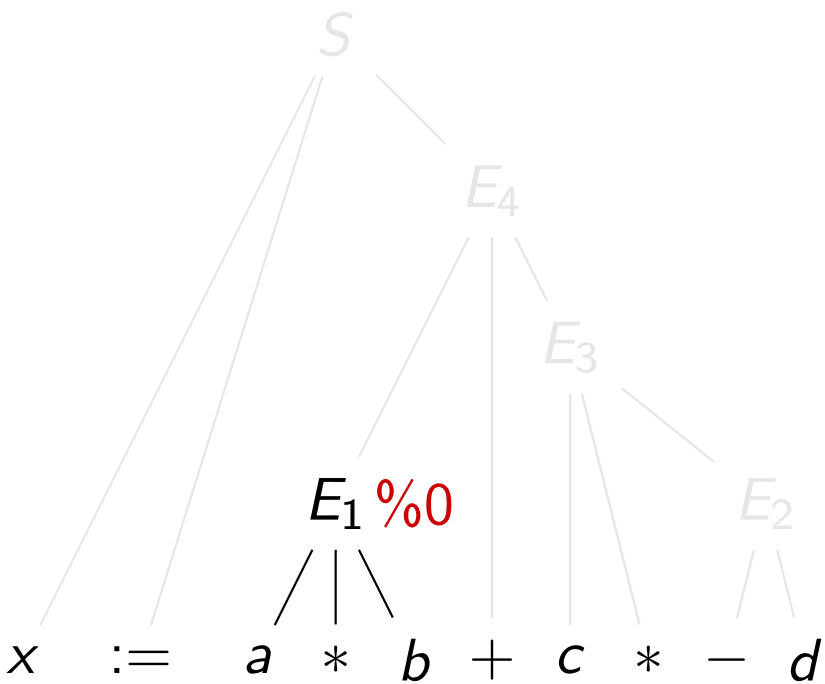
変換例

$$
\begin{array}{ll}
E_1 \to a * b & \{\%0 = a * b\} \\
E_2 \to -d & \{\%1 = -d\} \\
E_3 \to c * E_2 & \{\%2 = c * \%1\} \\
E_4 \to E_1 + E_3 & \{\%3 = \%0 + \%2\} \\
S \to x := E_4 & \{x = \%3\}
\end{array}
$$



変換例

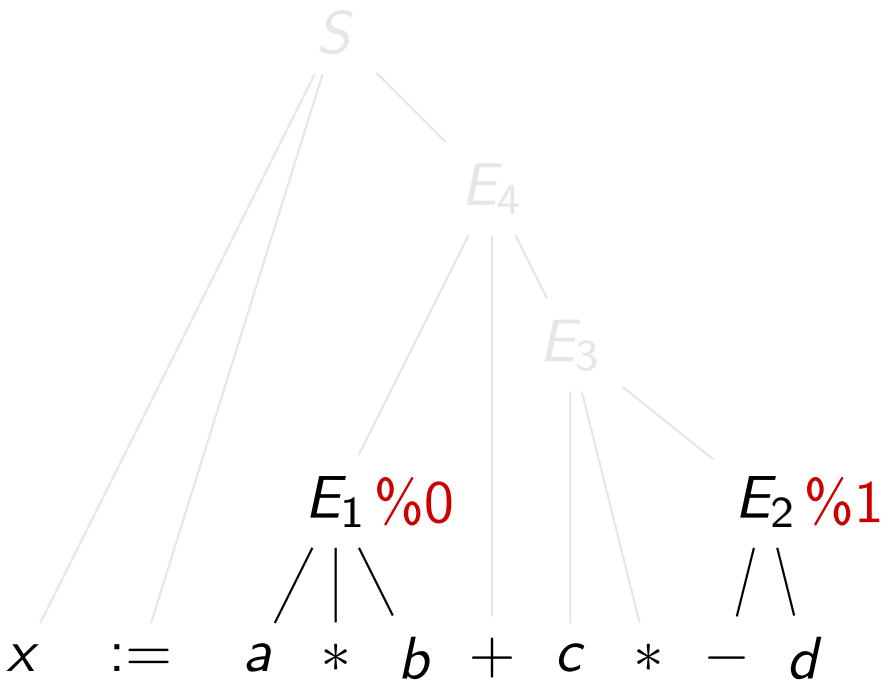
$$
\begin{array}{ll}
E_1 \to a * b & \{\%0 = a * b\} \\
E_2 \to -d & \{\%1 = -d\} \\
E_3 \to c * E_2 & \{\%2 = c * \%1\} \\
E_4 \to E_1 + E_3 & \{\%3 = \%0 + \%2\} \\
S \to x := E_4 & \{x = \%3\}
\end{array}
$$



$$\%0 = a * b$$

変換例

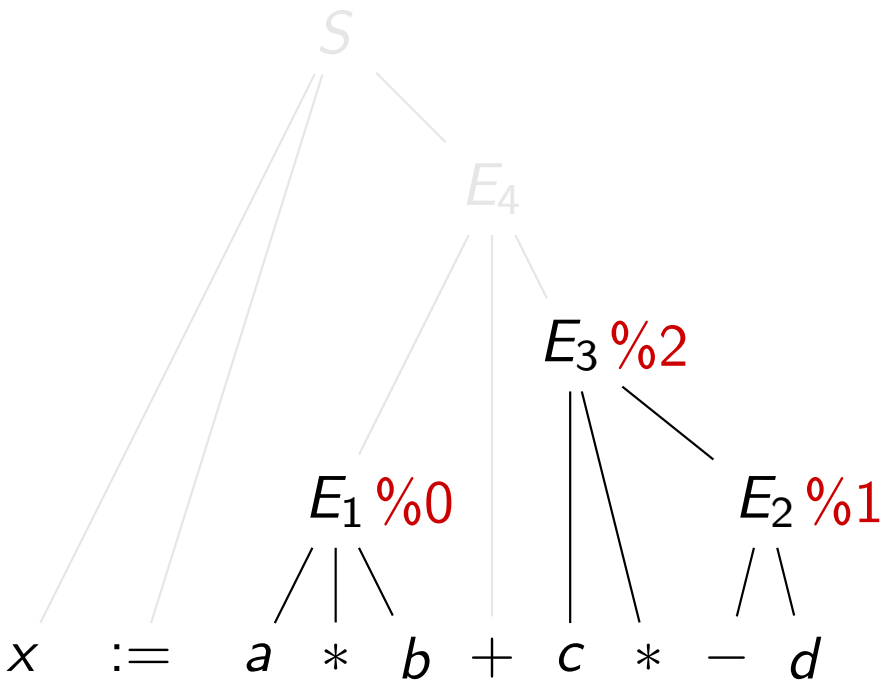
$$
\begin{array}{ll}
E_1 \to a * b & \{\%0 = a * b\} \\
E_2 \to -d & \{\%1 = -d\} \\
E_3 \to c * E_2 & \{\%2 = c * \%1\} \\
E_4 \to E_1 + E_3 & \{\%3 = \%0 + \%2\} \\
S \to x := E_4 & \{x = \%3\}
\end{array}
$$



$$
\begin{aligned}
\%0 &= a * b \\
\%1 &= -d
\end{aligned}
$$

変換例

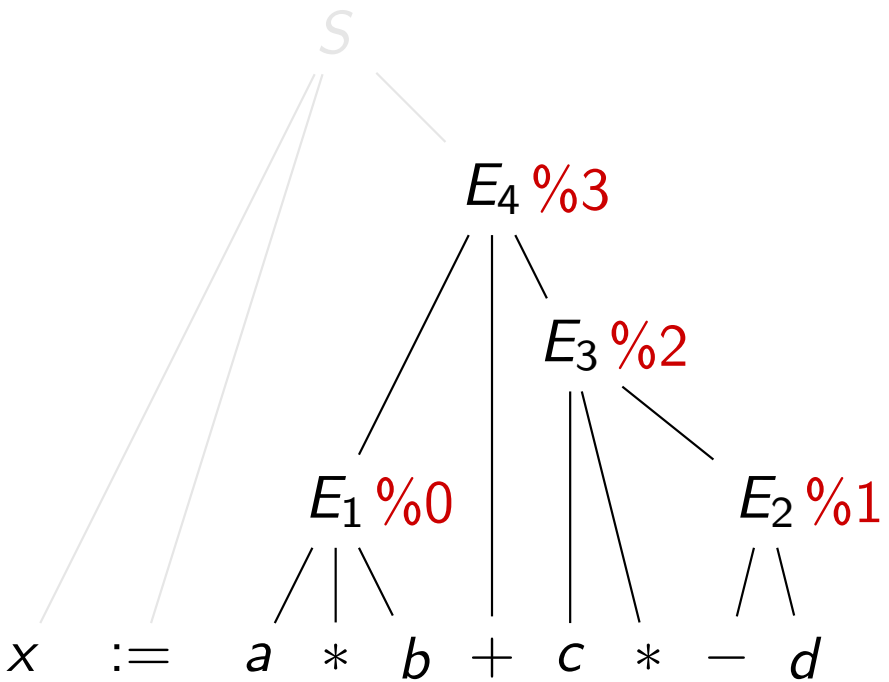
$$
\begin{array}{ll}
E_1 \to a * b & \{\%0 = a * b\} \\
E_2 \to -d & \{\%1 = -d\} \\
E_3 \to c * E_2 & \{\%2 = c * \%1\} \\
E_4 \to E_1 + E_3 & \{\%3 = \%0 + \%2\} \\
S \to x := E_4 & \{x = \%3\}
\end{array}
$$



$$
\begin{aligned}
\%0 &= a * b \\
\%1 &= -d \\
\%2 &= c * \%1
\end{aligned}
$$

変換例

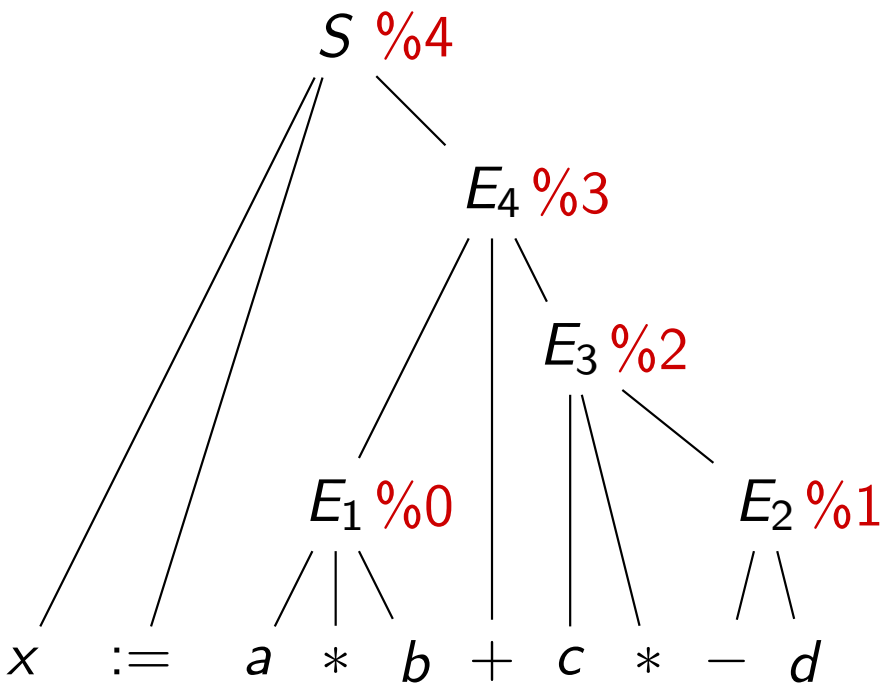
$$
\begin{array}{ll}
E_1 \to a * b & \{\%0 = a * b\} \\
E_2 \to -d & \{\%1 = -d\} \\
E_3 \to c * E_2 & \{\%2 = c * \%1\} \\
E_4 \to E_1 + E_3 & \{\%3 = \%0 + \%2\} \\
S \to x := E_4 & \{x = \%3\}
\end{array}
$$



$$
\begin{aligned}
\%0 &= a * b \\
\%1 &= -d \\
\%2 &= c * \%1 \\
\%3 &= \%0 + \%2
\end{aligned}
$$

変換例

$$
\begin{array}{ll}
E_1 \to a * b & \{\%0 = a * b\} \\
E_2 \to -d & \{\%1 = -d\} \\
E_3 \to c * E_2 & \{\%2 = c * \%1\} \\
E_4 \to E_1 + E_3 & \{\%3 = \%0 + \%2\} \\
S \to x := E_4 & \{x = \%3\}
\end{array}
$$

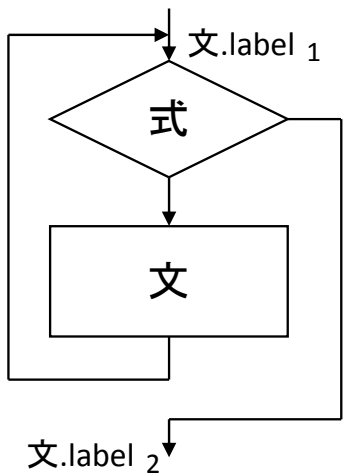


$$
\begin{aligned}
\%0 &= a * b \\
\%1 &= -d \\
\%2 &= c * \%1 \\
\%3 &= \%0 + \%2 \\
x &= \%3
\end{aligned}
$$

条件分岐に対する構文主導型変換

分岐条件の評価の後に分岐命令を生成する必要がある．

文	→	WHILE '('	{ 文.$label_1$:= newLabel();
			generate('goto 文.$label_1$')}
		式 ')'	{ 文.$label_2$:= newLabel();
			generate('if_not 式.var goto 文.$label_2$ ')}
		文$^{(1)}$	{generate('goto 文.$label_1$');
			generate('label 文.$label_2$')}



while 文の中間コード生成例

```
while(x<20) {
    r=r+x;
    x=x+1;
  }
```

$L1 :=$ `newLabel()`
`generate(`‘`label`$L1$’`)`
`generate(`‘$t1{:=}x < 20$’`)`
`generate(`‘`label`$L2$’`)`
`generate(`‘`if_not` $t1$ `goto` $L2$ ’`)`
`generate(`‘$t2{:=}r + x$’`)`
`generate(`‘$r{:=}t2$’`)`
`generate(`‘$t3{:=}x + 1$’`)`
`generate(`‘$x{:=}t3$’`)`
`generate(`‘`goto`$L1$’`)`
`generate(`‘`label`$L2$’`)`

```
L1:   t1:=x<20
      if not t1 goto L2
      t2:=r+x
      r:=t2
      t3:=x+1
      x:=t3
      goto L1)
L2:
```

上昇型構文解析におけるコード生成

IF 文

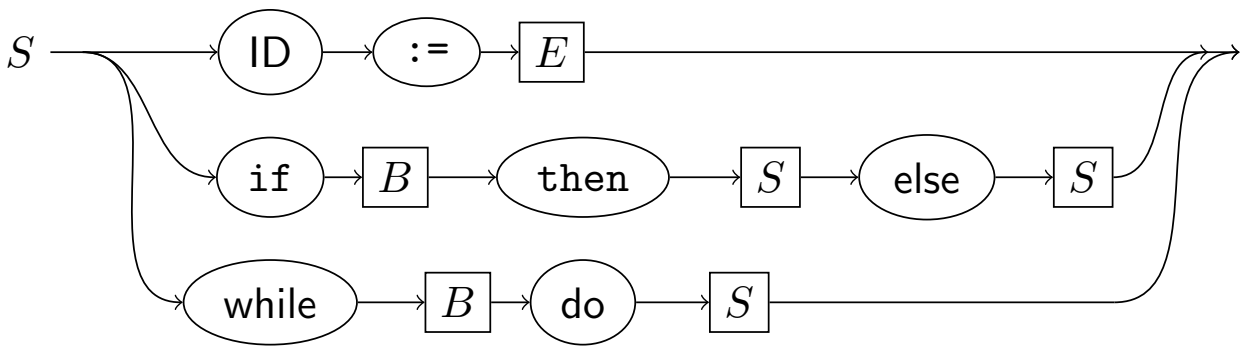
$$
\begin{array}{rcll}
I & \to & \texttt{if (}\ E\ \texttt{)} & \{I.label := \texttt{newLabel}();\\
 & & & \texttt{generate}(\text{`}\texttt{if_not}\ E.var\ \texttt{goto}\ I.label')\} \\
S' & \to & IS^{(1)} & \{S'.label := \texttt{newLabel}();\\
 & & & \texttt{generate}(\text{`}\texttt{goto}\ S'.label');\\
 & & & \texttt{generate}(\text{`}\texttt{label}\ I.label')\} \\
S & \to & S'\ \texttt{else}\ S^{(2)} & \{\texttt{generate}(\text{`}\texttt{label}\ S'.label')\}
\end{array}
$$

WHILE 文

$$
\begin{array}{rcll}
I & \to & \texttt{while (} & \{I.label := \texttt{newLabel}();\\
 & & & \texttt{generate}(\text{`}\texttt{label}\ I.label')\} \\
S' & \to & E\ \texttt{)} & \{S'.label := \texttt{newLabel}();\\
 & & & \texttt{generate}(\text{`}\texttt{if_not}\ E.var\ \texttt{goto}\ S'.label')\} \\
S & \to & IS'S^{(1)} & \{\texttt{generate}(\text{`}\texttt{goto}\ I.label');\\
 & & & \texttt{generate}(\text{`}\texttt{label}\ S'.label')\}
\end{array}
$$

下降型構文主導型変換

構文図式に従ってコードを生成する。
丸の部分でコードを生成する



```
// nextsym には先読みトークンがあらかじめ格納されている
If nexsym.kind = ID then                          // 代入文
    getNextsym()
    If nextsym.kind = ':=' then                   // 代入記号
        E のコードを生成 ($tempE に結果を格納)    // E
        generate('var(ID.val) = $tempE')          // 代入本体
    else error()
else if nextsym.kind = if then                    // 条件文
    getNextsym()
    B のコードを生成 ($tempB に結果を格納)        // B
    L1 = newlabel()                               // 偽の場合の飛び先
    generate('if_not $tempB goto L1')             // 分岐命令本体
    S のコードを生成                              // 条件が真の場合の S
    L2 = newlabel()                               // 偽の命令の後への飛び先
    generate('goto L2')
    generate('label L1')
    S のコードを生成                              // 条件が偽の場合の S
    generate('label L2')                          // 真の S のジャンプ先
else if nextsym.kind = while then                 // 繰り返し文
    getNextsym()
    L1 = newlabel()
    generate('label L1')                          // 繰り返しで戻るラベル
    B のコードを生成 ($tempB に結果を格納)        // B
    If nextSym.kind = do then
        L2 = newlabel()
        generate('if_not $tempB goto L2')
        S のコードを生成                          // 繰り返し本体の S
        generate('goto L1')                       // 繰り返しに戻る
        generate('label L2')                      // 脱出先のラベル
    else error()
else
    error()
```